UNIVERSITI MALAYA

KOD KURSUS	:	WIX1002
NAMA KURSUS	:	FUNDAMENTALS OF PROGRAMMING
NAMA PENSYARAH	:	DR UNAIZAH HANUM BINTI OBAIDELLAH
SEMESTER & SESI	:	SEMESTER 1, 2024/2025
KUMPULAN	:	OCC 1

BIL.	**NAMA**	**NO MATRIK**
1	AHMAD MUKHLIS BIN MOHD NORDIN	23002476
2	AZNUR ANAK JITWIL @ JIKUIL	23001792
3	MUHAMMAD AFIQ BIN MOHD FAUZI	23002465
4	MUHAMMAD SYAMEER HARITH BIN MOHD YUSOF	23002196
5	JOECHELE LIM QIU YING	24004535

Contents

QUESTION 1

Problem

In the fantasy world of Ravnora, usernames are not just identifiers, but they also hint at the nature of their owner. A powerful wizard named Eldranar has devised a method to determine if a username belongs to a friend or a foe based on the distinct magical runes in it. Eldranar's rule is simple:
if the number of distinct runes (characters) in a username is odd, then the user is considered a foe, otherwise a friend.

We code a program to help Eldranar determine if this user is a friend or a foe based on his method. The input will be a string type with only English alphabet.

Solution

In this code, we import Scanner class to be able to receive input from the user, which are the username of the person in that fantasy world. The user is prompt to input a lowercase name, but in case the user mistype the name, we use the function .toLowerCase() to ensure the program works fine.

Then, we create a final string variable that contains all the English alphabet in lowercase, so that we can compare the username with each of the alphabet.

We use nested for loop for this program, whereas the big loop will check each char from the final string, and the small loop will check for each character in the username input by the user.

In the small loop, we put an if statement to check for any char available. If one char appears, the counter will plus one. According to the rules, repetition of words are not counted, so we put a break statement inside if to cut the small loop.

After the program finished the loop, and if statement is use to check whether the username is an ally or an enemy. We use the condition total%2==0 to determine if the counter is even or odd. If its even, the program will output "ALLY DETECTED!" and if its odd, the output will be "ENEMY DETECTED".

Pseudocode

1. START
2. PRINT(Please input the username)
3. DEFINE FINAL string letter “abcdefghijklmnopqrstuvwxyz”
4. WHILE(true)
    4.1. DEFINE int total =0
    4.2. if(username.length()>100){
        4.2.1. PRINT(“Please input a valid username less than 100 letters");
        4.2.2. continue;
    4.3. IF(username.matches("[a-zA-Z]+"))
        4.3.1. FOR LOOP for (int i=length of the final string-1; i>=0; i--)
            4.3.1.1. FOR LOOP for (int j=length of the username-1; j>=0; j--)
                4.3.1.1.1. DEFINE char runes=username.charAt(i)
                4.3.1.1.2. DEFINE char temp=letter.charAt(j)
                4.3.1.1.3. IF(temp==runes)
                    4.3.1.1.3.1. total = total + 1
                    4.3.1.1.3.2. BREAK
                4.3.1.1.4. END IF
        4.3.2. END FOR
    4.4. ELSE
        4.4.1. PRINT(“Please input a valid username");
        4.4.2. continue;
    4.5. IF (total is even)
        4.5.1. PRINT(“ALLY DETECTED!)
    4.6. ELSE
        4.6.1. PRINT(ENEMY DETECTED!)
5. END WHILE
6. STOP

Sample Input and Output

```
Please input username
ahmad
ALLY DETECTED!
```

a-h-m-a-d
a+h+m+d = 4
4 is even
ahmad is an ally

```
Please input username
aphrodite
ENEMY DETECTED!
```

a-p-h-r-o-d-i-t-e
a+p+h+r+o+d+i+t+e=9
9 is odd
aphrodite is an enemy

```
Please input username
999
Please input a valid username
ahmadahmadahmadahmadahmadahmadahmadahmadahmadahmadah
Please input a valid username less than 100 letters
```

999 is not English alphabet
ahmadahmad….
username have more than 10 letters in it

Source code

```
Import java.util.Scanner;
public class Q1 {
   public static void main(String[] args){


      Scanner scanner = new Scanner(System.in);
      final String letter="abcdefghijklmnopqrstuvwxyz"; //26
      System.out.println("Please input username");
      while(true){
         String username = scanner.nextLine();//.toLowerCase();
         if(username.length()>100){
               System.out.println("Please input a valid username less than 100 letters");
               continue;
         }
         if(username.matches("[a-zA-Z]+")){
            int total=0;
            for(int i=letter.length()-1; i>=0; i--){ //25
               for(int j=username.length()-1; j>=0; j--){
```

```
                char runes = username.charAt(j);
                char temp = letter.charAt(i);

                if(temp==runes){
                    total++;
                    break;          // break for-loop j
                }
              }
            }
            if(total%2==0){
                System.out.println("ALLY DETECTED!");
            }else{
                System.out.println("ENEMY DETECTED!");
            }
        }else{
            System.out.println("Please input a valid username");
            continue;
        }
        break;
    }
  }
}
```

QUESTION 2

Problem

You are a teacher at a school that is holding a coding challenge as part of its annual Game Day event. The challenge for the participants is to create a program that simulates a classic game that reinforces number skills and logical thinking. The game is called "**LuluLala**".

In this game, participants have to write a program that asks user to enter a range of number (such as 1 as start and 100 as end) and prints numbers from 1 to 100. However, the rules for the game are as follows:

However, the rules for the game are as follows:

- If a number is a multiple of 3, the program should print "Lulu" instead of the number.
- If a number is a multiple of 5, the program should print "Lala" instead of the number.
- If a number is a multiple of both 3 and 5, the program should print "LuluLala".
- For all other numbers, the program should print the number itself.

Input Format: The program should prompt the user to enter two integers representing the start and end of a series of numbers.

Solution

Firstly, we need to import Scanner class so that the user can input two integer numbers: starting value and final value. After that, "**for**" loop is applied to iterate from starting value up to the end including final value. In addition, if-else statement is also included to check whether the number in this range is either a multiple of 3, or a multiple of 5 or both. If the number is a multiple of 3, the program displays "Lulu"; if the number is a multiple of 5, it displays "Lala"; and if the number is a multiple of 3 and 5, it displays "LuluLala". For the number not multiples of 3 and 5, the program displays the number itself.

Pseudocode

```
Begin
        display "Enter the starting value: "
        read startValue
        display "Enter the ending value: "
        read finValue
        display "The output is: "
        for (i = startValue; i <= finValue; i++)
                if (i % 3 == 0 && i%5 ==0)
                        display "LuluLala"
                else if (i%3 == 0)
                        display "Lulu"
                else if (i%5 == 0)
                        display "Lala"
                else
                        display i
                end if
        end for
End
```

Sample Input and Output

Example 1:

```
run:
Enter the starting value: 1
Enter the ending value: 20

The output is:
1
2
Lulu
4
Lala
Lulu
7
8
Lulu
Lala
11
Lulu
13
14
LuluLala
16
17
Lulu
19
Lala
BUILD SUCCESSFUL (total time: 3 seconds)
```

Example 2:

```
run:
Enter the starting value: 2
Enter the ending value: 15

The output is:
2
Lulu
4
Lala
Lulu
7
8
Lulu
Lala
11
Lulu
13
14
LuluLala
BUILD SUCCESSFUL (total time: 7 seconds)
```

Example 3:

```
run:
Enter the starting value: 15
Enter the ending value: 30

The output is:
LuluLala
16
17
Lulu
19
Lala
Lulu
22
23
Lulu
Lala
26
Lulu
28
29
LuluLala
BUILD SUCCESSFUL (total time: 5 se
```

Source Code

```
import java.util.Scanner;
public class VivaQuestion2 {
	public static void main(String[] args) {

		Scanner input = new Scanner(System.in);

		System.out.print("Enter the starting value: ");
		int startValue = input.nextInt();
		System.out.print("Enter the ending value: ");
		int finValue = input.nextInt();

		System.out.println("\nThe output is: ");

		for (int i = startValue; i <= finValue; i++) {
				if (i % 3 == 0 && i%5 ==0) {
					System.out.println("LuluLala");
				} else if (i%3 == 0) {
					System.out.println("Lulu");
				} else if (i%5 == 0) {
					System.out.println("Lala");
				} else
					System.out.println(i);
		}
		input.close();
	}
}
```

QUESTION 3

Problem Statement

A triangle is defined by three angles. For a triangle to be valid:

1. The sum of all three angles must equal 180°.
2. Each angle must be greater than 0°.

Write a Java program that takes three angles as input and determines if the triangle is valid or not based on the conditions mentioned. Determines the type of triangle based on the angles if the triangle is valid.

Triangle Type Rules:

1. Equilateral Triangle: All three angles are equal.

2. Isosceles Triangle: Any two angles are equal.

3. Scalene Triangle: All three angles are different.

4. Right-Angled Triangle: One of the angles is exactly 90°.

Solution

Firstly, we need to import Scanner class for user to input three angels A, B and C. We need to prompt user to input the angles in a “do-while loop” so the user can use the programs many times. In the same loop, after receives inputs from user, we need to check if sum of all the angles is equal to 180 and all the angles are greater than zero by using “if” statement. If these conditions are true, we will print “Valid triangle” and if the vice versa, we print “Invalid triangle”. For valid triangle, we use multiple “if” statements to tell the user the types of triangles based on the angles that they input. If one of the angles is equal to 90, we display it as right-angle triangle. If all the angles are equal to 60, we display it as an equilateral triangle but if any two angles are equal, we display it as isosceles triangle. If all three angles are different, we display it as scalene triangle. The loop stops when the user input negative angles.

Pseudocode

1. Start
2. Do-while loop

    2.1 Display “Enter three angles [-ve] angle to quit: ”

    2.2 Read angles input from user as A, B and C

    2.3 If (A < 0 || B < 0 || C < 0), break the loop

    2.4 Declare variable sum as : sum = A+B+C

    2.5 If (sum == 180 && A > 0 && B > 0 && C > 0),
    display “Triangle is valid”

    2.5.1 If (A == 90 || B == 90 || C == 90), display “right-angle triangle”

    2.5.2 If (A == 60 && B == 60 && C == 60), display “equilateral triangle”

    Else if (A == B || A == C || B == C), display “isosceles triangle”

    2.5.3 If (A != B && A != C && B != C), display “scalene triangle”.

    Else, display “Triangle is not valid”

End do–while loop

3. End

Sample input and output

```
Enter three angles [-ve angle to quit]: 60 60 60
The triangle is valid.
It is an equilateral triangle.
Enter three angles [-ve angle to quit]: 90 45 45
The triangle is valid.
It is a right-angled triangle.
It is an isosceles triangle.
Enter three angles [-ve angle to quit]: 90 35 55
The triangle is valid.
It is a right-angled triangle.
It is scalene triangle.
Enter three angles [-ve angle to quit]: 120 40 20
The triangle is valid.
It is scalene triangle.
Enter three angles [-ve angle to quit]: 100 80 45
The triangle is not valid.
Enter three angles [-ve angle to quit]: -1 -1 -1
------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------
```

Source Code

```
import java.util.Scanner;
public class VivaQ3 {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int angleA, angleB, angleC;

        do{
        System.out.print("Enter three angles [-ve angle to quit]: ");
        angleA = sc.nextInt();
        angleB = sc.nextInt();
        angleC = sc.nextInt();

        if(angleA < 0 || angleB < 0 || angleC < 0){
            break;
        }

        int sumAngle = angleA + angleB + angleC;
            if(sumAngle == 180 && angleA > 0 && angleB > 0 && angleC > 0){
                System.out.println("The triangle is valid.");

                if(angleA == 90 || angleB == 90 || angleC == 90){
                    System.out.println("It is a right-angled triangle.");
                }

                if(angleA == 60 && angleB == 60 && angleC == 60){
                    System.out.println("It is an equilateral triangle.");

                }else if(angleA == angleB || angleA == angleC || angleB == angleC){
                    System.out.println("It is an isosceles triangle.");
```

```
            }

            if(angleA != angleB && angleA != angleC && angleB != angleC){
                System.out.println("It is scalene triangle.");
            }


        }else{
            System.out.println("The triangle is not valid.");
        }
    }while(!(angleA < 0 || angleB < 0 || angleC < 0));
  }
}
```

QUESTION 4

Problem Statement

Input validation is a simple yet very important part of programming. Write a program that asks the user to input a positive integer. If the user does not comply (by inputting text, floating values, or negative integers), inform the user that their input is invalid, and reprompt the user to input a valid positive integer until the user complies. When a valid integer is inputted, declare if it is even or odd. We use input.charAt(i) when looping through the inputted text, where "input" is the variable where the input is stored.

Solution

- Firstly, we must declare an empty String named input so that the user's input can be received.
- After that,we need to declare a boolean named isaninteger to track if a positive integer has been inputted by user.
- Then, the program will continue into the do-while loop. In the do-while loop, the user's input will be read as a String. The boolean isaninteger will be initialized to true.
- Next, in the do-while loop, a for loop that has an if statement will be entered that will loop for each character in the String input to make sure that every character in input is a digit between '0' and '9'.
- If any of the character is not a digit, the boolean isaninteger will be set to false which means that the user has entered an invalid input. When an invalid character is met, the program will exit the for loop because of the break statement.
- The do-while loop will continue until a valid input is entered because of the condition !isaninteger. After a valid input is entered, the boolean isaninteger remains true since it doesn't match the condition in the if statement. The program then will exit the do-while loop.
- After the do-while loop, we must convert the String input to an integer, number because the program cannot do mathematical calcultions with String.
- It will check if number is even by using int number%2==0. If it is true, then the program will print that the integer is valid an even. If it is false, the program will print the integer is valid and odd.

Pseudocode

1. Start
2. Declare String input=” “;
3. Declare boolean isaninteger;
4. Print (“ Please input a positive integer”);
5. do {

   5.1 User input a string

   5.2 Declare isaninteger = true;

   5.3 for (int i=0; i<input.length(); i++){

      5.3.1 if (!(input.charAt(i)>= ‘0’ && input.charAt(i) <= ‘9’))

      5.3.2 isaninteger = false;

      5.3.3 Print (“Invalid input. Please re-input a valid +ve integer : ”);

      5.3.4 break;

   }while(!isaninteger);
6. Declare int number= Integer.parseInt(input);
7. If (number%2==0)

   7.1 Print (“you’ve inputed a valid integer!”)

   7.2 Print (“The integer is even!”)

8. else

   8.1 Print (“you’ve inputed a valid integer!”)

   8.2 Print (“The integer is odd!”)

9. End

Sample input and output

```
Please input a positive integer : five
Invalid input. Please re-input a valid +ve integer : -20
Invalid input. Please re-input a valid +ve integer : 9.2
Invalid input. Please re-input a valid +ve integer : 18
you've inputed a valid integer!
The integer is even!
BUILD SUCCESSFUL (total time: 38 seconds)
```

```
Please input a positive integer : 21ab3
Invalid input. Please re-input a valid +ve integer : 51!
Invalid input. Please re-input a valid +ve integer : 16%
Invalid input. Please re-input a valid +ve integer : 9
you've inputed a valid integer!
The integer is odd!
BUILD SUCCESSFUL (total time: 22 seconds)
```

Source Code

```
package vivaq4;
import java.util.Scanner;
public class VivaQ4 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String input="";
        boolean isaninteger;
        System.out.print(" Please input a positive integer : ");
        do{
          input = scanner.next();
          isaninteger = true;
          for (int i = 0; i < input.length(); i++) {
            if (!(input.charAt(i) >= '0' && input.charAt(i) <= '9')){
              isaninteger = false;
              System.out.print("Invalid input. Please re-input a valid +ve integer : ");
              break;        // Break for-loop
```

```
            }
        }
    }while(!isaninteger);
    int number = Integer.parseInt(input);     // Convert String to int
    if (number % 2 == 0) {
        System.out.println("you've inputed a valid integer!");
        System.out.println("The integer is even!");
    } else {
        System.out.println("you've inputed a valid integer!");
        System.out.println("The integer is odd!");
    }
  }
}
```

QUESTION 5

Problem

We were given a positive integer n. The task is to group the digits of nnn into groups such that each group contains the same digit repeatedly, and each digit group is as long as possible. The digit groups should be in the order of appearance in nnn.

For example, if n=1223334444, you can split it into "1", "22", "333", "4444". Once the digit groups are formed, your task is to determine:

1. The total number of digit groups.
2. The digit that forms the longest group and its length.
3. The sum of lengths of all digit groups. If there is a tie for the longest group (multiple groups have the same length), choose the digit that appears first.

Solution

Firstly, the Scanner class is imported to allow users to input a positive integer, n. Next, the program starts by checking if the input is positive. If n is less than or equal to 0, it prints "Invalid input", otherwise it proceeds with further steps. Then, the integer n is converted into a string so that it can be easily manipulated, as strings allow character-by-character inspection. The program iterates through the string representation of the number and counts how many "groups" of consecutive identical digits there are. A "group" is defined as a sequence of consecutive digits that are the same.

For example, for the number 122233, the groups are:

Group 1: 1

Group 2: 222

Group 3: 33

The program then counts and outputs the total number of such groups. If there is more than one group, the program finds the longest group. For each group, it counts how many times the digit repeats. If it encounters a group longer than the current longest one, it updates the longest group's length and the digit that repeats the most. The program keeps track of the digit that forms the longest group, and the length of that group. For example, if the number is 1222333, the longest group is 333 with length 3. However, if there is only one group (i.e., all the digits in the number are the same), the program simply prints the repeated digit and the total number of digits in the number. The program outputs the number of groups and, if applicable, the digit that occurs in the longest group along with its count. Lastly, it also prints the total length of the number, which is the number of digits.

Pseudocode

1. Get the positive integer from the user as variable long.
2. If the input is not a positive integer
    a. Print invalid input
3. Else
    a. Convert the input into string
    b. Get the length of the string
    c. Initialize counter i and j
    d. Initialize number of groups, length of groups and length of the longest groups
    e. Initialize the digit of the longest group
    f. While counter i is less than the length of the string minus 1
        i. If the current digit is not the same as the next digit
            1. Number of groups plus 1
        ii. Add 1 to counter i
    g. Print number of groups
    h. If number of groups is not 1
        i. While counter j is less than the length of the string
            1. If current digit is the same as the digit before
                a. Add 1 to length of groups
            2. If the length of groups is more than the length of the longest group
                a. The digit of the longest group is set to the current digit
                b. The length of the longest group is set to the current group length
            3. If the current digit is not the same as the digit before
                a. The length of groups is set to 1
            4. Add 1 to counter j
        ii. Print the digit of the longest group and its length
    i. Else
        i. Print the current digit and the length of the string
    j. Print the length of the string

Sample Input and Output

1.

```
Enter your integer : 0011223334
4
3 3
8
```

2.

```
Enter your integer : 00001222233344
4
2 4
10
```

3.

```
Enter your integer : 43332441111
5
1 4
11
```

Source Code

```
import java.util.Scanner;
public class VivaQ5 {

    public static void main(String[] args) {
        Scanner in=new Scanner(System.in);
        long n;
        System.out.print("Enter your integer : ");
        n=in.nextLong();
        if(n<=0){
            System.out.println("Invalid input");
        }
        else{
            String nString=Long.toString(n);
            int length=nString.length();          // Total length of digits
            int i=0,j=1;
            int group=1,count=1,max=0;
            char maxDigit=' ';
            while(i<length-1){ //count how many groups
                if(nString.charAt(i)!=nString.charAt(i+1)){
                    group++;
                }
                i++;
            }
            System.out.println(group);
            if(group!=1){
                while(j<length){ //count the length of groups
                    if(nString.charAt(j)==nString.charAt(j-1)){
                        count++;
                    }
                    if(count>max){
                        maxDigit=nString.charAt(j-1);
                        max=count;
                    }
                    if(nString.charAt(j)!=nString.charAt(j-1)){
                        count=1;
                    }
                    j++;
                }
                System.out.println(maxDigit + " " + max);
            }
            else{
                System.out.print(nString.charAt(j)+" ");
```

```
            System.out.println(length);
        }
        System.out.println(length);
    }

}
}
```

When group = 1, length of group = length of all digits

QUESTION 6

Problem

Write a Java program that prints the word "MALAYSIA" in a specific star (*) pattern format. Each letter should be represented in a 7x7 grid, following the star patterns shown below.

Pattern Specifications:

1. Each letter in "MALAYSIA" is printed in a 7x7 grid.
2. Use * characters to form each letter, with appropriate spacing between the letters.
3. Use loops and conditional statements to construct each letter's pattern within the main loop.
4. Print the letters horizontally with spaces between them to keep the word aligned.

Solution

We create a for loop that will loop 7 times following the patterns specification. The for loop represent a row. Next, inside the for loop, we create an if statement to print every letter, row by row. The condition would be something like, if(i == 0) whereas variable i is the row.

Pseudocode

```
1. START
2. FOR LOOP for(int i =0; i<7 ; i++)
3. PRINT M
    3.1. IF(i==0 || i==4 || i==5 || i==6)
        3.1.1.  PRINT(*     *)
    3.2. IF(i==1)
        3.2.1.  PRINT(**   **)
    3.3. IF(i==2)
        3.3.1.  PRINT(* * * *)
    3.4. IF(i==3)
        3.4.1.  PRINT(*  *  *)
    3.5. PRINT(  )
4. PRINT A
    4.1. IF(i==0 || i==3)
        4.1.1.  FOR LOOP for(int j=0; j<7; j++)
            4.1.1.1.    PRINT(*)
        4.1.2.  ELSE
            4.1.2.1.    PRINT(*     *)
        4.1.3.  PRINT("  ")
5. PRINT L
    5.1. IF(i==6)
        5.1.1.  FPR LOOP for(int j=0; j<7; j++)
            5.1.1.1.    PRINT(*)
    5.2. ELSE
        5.2.1.  PRINT(*      )
    5.3. PRINT("  ");
```

6. PRINT A
7. PRINT Y
    7.1. IF(i==0)
        7.1.1. PRINT(* *)
    7.2. IF(i==1)
        7.2.1. PRINT(* *)
    7.3. IF(i==2)
        7.3.1. PRINT(* *)
    7.4. IF(i==3 || i==4 || i==5 || i==6)
        7.4.1. PRINT(*)
    7.5. PRINT(" ")
8. PRINT S
    8.1. IF(i==0 || i==3 || i==6)
    8.2. FOR LOOP for(int j=0; j<7; j++)
            8.2.1.1. PRINT(*)
        8.2.2. ELSE IF(i==1 || i==2)
            8.2.2.1. PRINT(*)
        8.2.3. ELSE
            8.2.3.1. PRINT(*)
    8.3. PRINT( )
9. PRINT I
    9.1. IF(i==0 || i==6)
        9.1.1. FOR(int j=0; j<7; j++)
            9.1.1.1. PRINT(*)
    9.2. ELSE
        9.2.1. PRINT(*)
    9.3. PRINT( )
10. PRINT A
11. END FOR
12. STOP

Source code

```
public static void main(String[] args){
    for(int i=0; i<7; i++){
    //M
    if(i==0 || i==4 || i==5 || i==6)
        System.out.print("*     *");
    if(i==1)
        System.out.print("**   **");
    if(i==2)
        System.out.print("* * * *");
    if(i==3)
        System.out.print("*  *  *");
    System.out.print("  ");9

    //A
    if(i==0 || i==3)
        for(int j=0; j<7; j++)
            System.out.print("*");
    else
        System.out.print("*     *");
    System.out.print("  ");

    //L
    if(i==6)
        for(int j=0; j<7; j++)
            System.out.print("*");
    else
        System.out.print("*      ");
    System.out.print("  ");

    //A
    if(i==0 || i==3)
        for(int j=0; j<7; j++)
            System.out.print("*");
    else
        System.out.print("*     *");
    System.out.print("  ");

    //Y
    if(i==0)
        System.out.print("*     *");
    if(i==1)
        System.out.print(" *   * ");
```

```
    if(i==2)
        System.out.print("  * *  ");
    if(i==3 || i==4 || i==5 || i==6)
        System.out.print("   *   ");
    System.out.print("  ");

    //S
    if(i==0 || i==3 || i==6)
        for(int j=0; j<7; j++)
            System.out.print("*");
    else if(i==1 || i==2)
        System.out.print("*      ");
    else
        System.out.print("      *");
    System.out.print("  ");

    //I
    if(i==0 || i==6)
        for(int j=0; j<7; j++)
            System.out.print("*");
    else
        System.out.print("   *   ");
    System.out.print("  ");

    //A
    if(i==0 || i==3)
        for(int j=0; j<7; j++)
            System.out.print("*");
    else
        System.out.print("*     *");
    System.out.print("  ");

    //move to next line
    System.out.println();
    }
}
```

Sample input and output

```
run:
*      *   *******   *         *******   *      *   *******   *******   *******
**    **   *      *   *         *      *    *    *    *            *      *      *
* * * *   *      *   *         *      *     * *     *            *      *      *
*   *   *   *******   *         *******      *      *******      *      *******
*       *   *      *   *         *      *      *            *      *      *      *
*       *   *      *   *         *      *      *            *      *      *      *
*       *   *      *   *******   *      *      *      *******   *******   *      *
BUILD SUCCESSFUL (total time: 0 seconds)
```